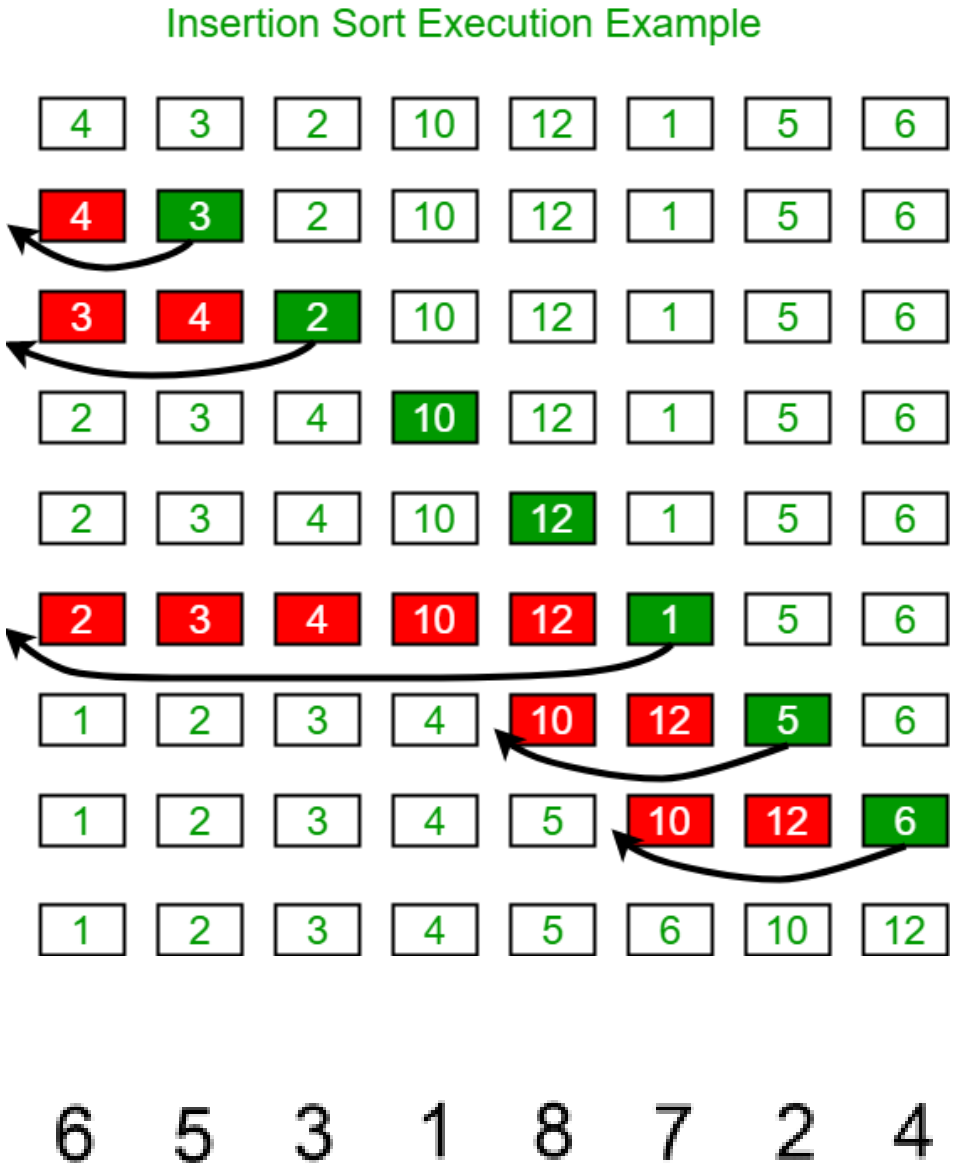
# C Programming
# Sorting

# Pre-learning

- If you have not watched the following video, please do so before tackling the Exercise.
  - https://wsdmoodle.waseda.jp/mod/millvi/view.php?id=5822906

# Insertion Sort

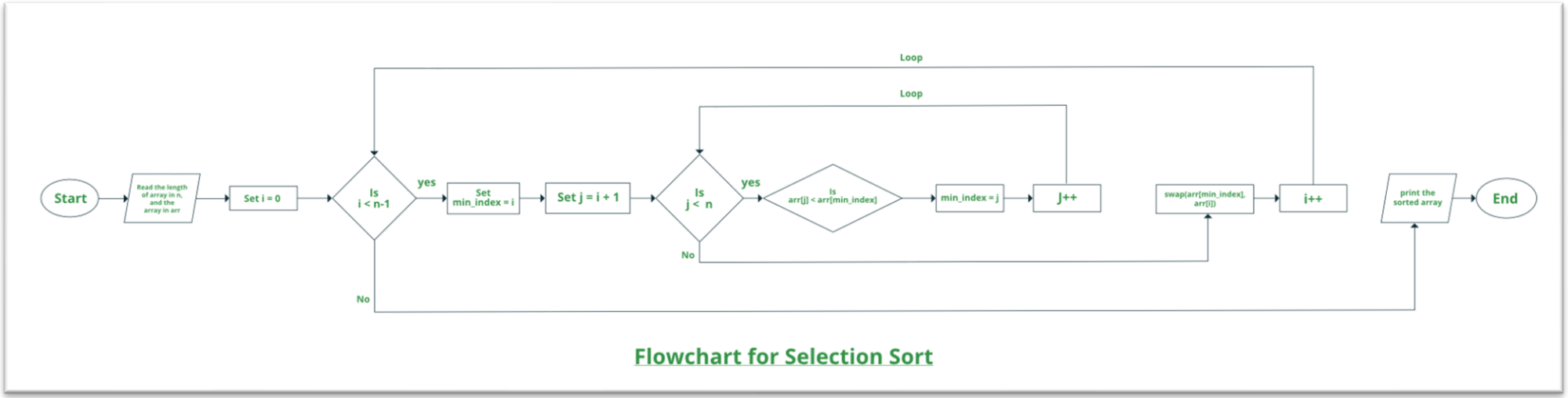
- Insertion sort is a simple sorting algorithm that works similarly to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed in the correct position in the sorted part.




Reference: https://www.geeksforgeeks.org/insertion-sort/

# Selection Sort

Selection sort is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and moving it to the sorted portion of the list.

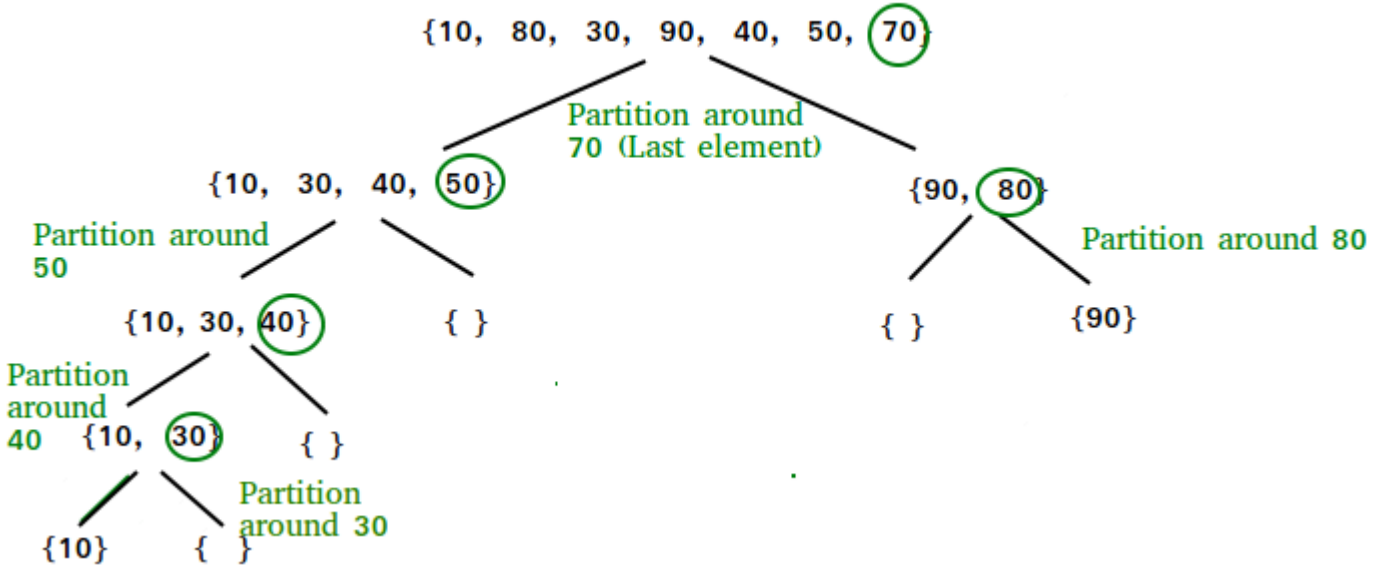



Flowchart for Selection Sort

Reference: https://www.geeksforgeeks.org/selection-sort/

# QuickSort

QuickSort is a sorting algorithm based on the Divide and Conquer algorithm that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.




Reference: https://www.geeksforgeeks.org/quick-sort/

# Find out on your own about the following algorithms

- Heap Sort
- Bubble Sort

# Exercises
# Exercise 1 - 5

- Store 10 random numbers from 1 to 100 in an array and sort them in order of decreasing number using the following sorting method.
  1. Insertion Sort
  2. Selection Sort
  3. Quick Sort
  4. Heap Sort
  5. Bubble Sort
- Create a program for each.
- Separate programs for each sort.

# Random Number Generation Sample

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void main()
{
    int i, a[29];
    int rn;

    srand((unsigned) time(NULL));

    for(i=0;i<10;i++)
    {
        rn=rand()%100 + 1;
        a[i] = rn;
        printf("%d, ",a[i]);
    }
}
```

# Exercise 6 - 7

- Ex6: Store 10 random numbers (1–100) in an array, sort them in descending order, and display only the values greater than or equal to 50 (using selection sort).
  - Extract the values “50 or greater” from the original array.
  - Sort them in descending order using selection sort.
  - Display both the original array and the sorted array.
- Ex7: Store 10 random numbers (1–100) in an array, sort only the even numbers in ascending order, and display the result (using bubble sort).
  - Store 10 random numbers in the array.
  - Extract only the even numbers and place them in a separate array.
  - Sort them in ascending order using bubble sort.
  - Display both the original array and the sorted array.

# Exercise 8

- Ex 8: Store 10 random numbers (1–100) in an array, and compare the execution times of bubble sort and quick sort.
  - Use `clock()` to measure the time.
  - Copy the same array and use it for both sorting algorithms.
  - Display the execution time (in milliseconds).

## Example of a program that measures execution speed

```
#include <stdio.h>
#include <time.h>
int main(void)
{
    clock_t start_time, end_time;
    start_time = clock();

    //Write the process you want to measure here

    end_time = clock();
    printf("time: %f¥n",(double)(end_time-start_time));
    #printf("time: %f¥n",(double)(end_time-start_time)/CLOCKS_PER_SEC);

    return (0);
}
```

# About Submitting Exercises

- Summarize the exercises you worked on in Word.
  - Source code
  - Screenshot of execution result
  - Program Description
- Submission is not mandatory. However, these exercises need to be summarized in a report at Lecture 9:Report 2. Therefore, please be sure to work towards Lecture 9.

# Tips:
# computational complexity

Sorting algorithms have different characteristics, and choosing the right one depends on the context.

There are differences in the amount of calculation, etc.

An index that expresses the arithmetic performance of an algorithm as a percentage of the increase in execution time relative to the increase in data volume.

Worst Case:

- Insertion Sort: $O(n^2)$
- Selection Sort: $O(n^2)$
- Quick Sort: $O(nlog(n))$
- Heap Sort: $O(nlog(n))$
- Bubble Sort: $O(n^2)$

**You can do some research on it if you are interested**